FORM-TL-01 · Vecindario
Notas de presentación: bloque de publicación y muro

APUNTE PAZMIÑO WASHINGTON GERMAN

Periodo académico 2026–2027

1. Publicación y distribución al muro

Recorrido de un aviso. El usuario o el negocio local crea el aviso desde el cliente (CLI). La primera llamada (S1) envía a PUB el `id_usuario`, el texto, la categoría, la ubicación y el tipo de aviso (normal o alerta); PUB lo registra y responde con el `id_aviso`. Si el aviso lleva imagen, el cliente la sube directamente al servicio externo de almacenamiento (S2), que devuelve la URL o el identificador del archivo, y después registra esa referencia en PUB (S2b). Así el archivo no pasa por los servicios internos.

Cuando el aviso está completo, PUB publica el evento `AvisoPublicado` en Kafka (A1) y no espera nada más: el usuario ya recibió su confirmación. MUR consume el evento (A2) con un *planificador*, que pide a SEG los seguidores del autor por páginas mediante gRPC (S3) y publica un lote por página (`id_aviso`, número de página) en el tópico `muro.lotes`. Los *trabajadores* de MUR toman esos lotes en paralelo y agregan el aviso al muro de cada seguidor. En paralelo, NOT consume el evento `NuevaPublicación` o `AlertaSeguridad` (A3) y envía la notificación push a los seguidores (A5). Cuando un seguidor abre la aplicación, obtiene de MUR su muro ya armado (S10).

ID	Origen → destino	Tipo	Sincronía	Protocolo y middleware
S1	CLI → PUB	Petición-respuesta	Síncrona	HTTPS/TCP, REST API
S2	CLI → almac. externo	Petición-respuesta	Síncrona	HTTPS/TCP, API externa
S2b	CLI → PUB	Petición-respuesta	Síncrona	HTTPS/TCP, REST API
A1	PUB → Kafka	Mensajería	Asíncrona	TCP, Kafka
A2	Kafka → MUR (lotes)	Mensajería	Asíncrona	TCP, Kafka
S3	MUR → SEG	RPC	Síncrona	HTTP/2, gRPC
A3	Kafka → NOT	Mensajería	Asíncrona	TCP, Kafka
A5	NOT → CLI	Mensajería push	Asíncrona	HTTPS/TCP, FCM
S10	CLI → MUR	Petición-respuesta	Síncrona	HTTPS/TCP, REST API

Mecanismos y justificación.

- **REST sobre HTTPS, síncrono (S1, S2, S2b).** Crear el aviso, subir la imagen y vincularla son acciones en las que el usuario necesita una respuesta inmediata (`id_aviso` o URL) para continuar. HTTPS funciona de forma nativa en Android y en el navegador y protege los datos en tránsito.
- **Kafka, asíncrono (A1, A2, A3).** La distribución al muro y las notificaciones pueden completarse después de responder al usuario. Kafka desacopla a PUB de MUR y de

NOT: PUB publica el evento y continúa, y el evento queda guardado aunque un consumidor esté caído.

- **gRPC sobre HTTP/2, síncrono (S3).** MUR necesita cada página de seguidores para seguir distribuyendo. gRPC trabaja con mensajes binarios compactos y un contrato `.proto` entre servicios internos, por lo que tiene menos latencia que REST con JSON.

R3: tiempos distintos para avisos normales y alertas. Tomamos los tiempos de R3 como objetivos de diseño: hasta 5 s para un aviso normal, con consistencia eventual aceptable, y menos de 1 s para una alerta de seguridad. Para que una alerta no espere detrás de los avisos normales, PUB la publica en un tópico propio (`alertas.seguridad`), que leen grupos de consumidores dedicados en MUR y en NOT. Si se acumulan avisos normales en `avisos.publicados`, esa cola no retrasa a las alertas, porque tienen su propio tópico y su propia capacidad de procesamiento.

R4: un negocio con 30.000 seguidores. PUB no espera a que el aviso llegue a los 30.000 seguidores, porque la distribución es asíncrona (A1). Un evento de Kafka lo procesa un solo consumidor del grupo, así que no se reparte solo entre varios. Por eso el planificador de MUR lo divide: pide los seguidores a SEG por páginas (S3) y publica un lote por página en `muro.lotes`. Ese tópico tiene varias particiones, y los trabajadores de MUR, que forman un mismo grupo de consumidores, se reparten los lotes y los procesan en paralelo. Además, se limita cuántos lotes de un mismo autor se procesan a la vez, para que una publicación masiva no ocupe toda la capacidad y el servicio siga respondiendo al resto de usuarios.

F2: el servicio de muro se cae mientras un negocio publica. La distribución no se pierde, porque la unidad que se confirma y se reintenta es el lote. Cada trabajador confirma su lote en Kafka (*commit* del *offset*) solo después de escribir el aviso en todos los muros de ese lote, y el planificador confirma el `AvisoPublicado` solo cuando ya publicó todos los lotes. Si MUR se cae a mitad del trabajo, al recuperarse se vuelven a leer únicamente los lotes que no se confirmaron. Como un lote puede procesarse dos veces, la escritura es idempotente: agregar el mismo aviso al mismo muro no crea un duplicado, gracias a la clave única (`id_usuario`, `id_aviso`).

Socket TCP para la práctica PE-U1 (propuesta del equipo). WSS funciona sobre TCP, pero añade el protocolo WebSocket encima. Para dejar clara la interacción con sockets que pide la guía, el prototipo de PE-U1 implementará la mensajería directa (S4) con un socket TCP directo: cada mensaje tendrá un formato fijo (`message_id`, `sequence_no` y contenido) y el servidor responderá con un ACK. La consulta de seguidores (S3) se implementará con gRPC. Así se podrán comparar en la práctica la latencia y el tamaño de los mensajes de las dos formas. En producción, S4 seguirá usando WSS, porque el navegador no permite abrir sockets TCP directos y WSS viaja cifrado por el mismo puerto que HTTPS.

2. Guion de exposición

Tramo de Washington: de 0:40 a 1:50 (unos 70 segundos).

1. **Publicar con imagen (S1, S2, S2b), unos 15 s.** El cliente crea el aviso en PUB por REST y recibe su identificador. La imagen va directo al almacenamiento externo, que devuelve la URL, y S2b la liga al aviso. Son llamadas síncronas porque el usuario necesita la confirmación.

2. **Kafka (A1), unos 10 s.** Desde aquí todo es asíncrono: PUB publica `AvisoPublicado` en Kafka y no espera, así que publicar no se vuelve lento aunque el autor tenga 30.000 seguidores.

3. **Muro, gRPC y lotes (A2, S3), unos 15 s.** El planificador de MUR pide a SEG los seguidores por páginas con gRPC y crea un lote por página en `muro.lotes`. Los trabajadores de MUR toman esos lotes en paralelo y actualizan los muros.

4. **Alertas, R3, unos 10 s.** Las alertas de seguridad van por un tópico propio con consumidores dedicados, así que no esperan detrás de los avisos normales. Nuestro objetivo de diseño es menos de un segundo; un aviso normal puede tardar hasta cinco.

5. **Notificaciones (A3), unos 5 s.** NOT escucha el mismo evento y notifica a los seguidores.

6. **Fallo F2, unos 15 s.** Si MUR se cae a mitad de la distribución, no se pierde nada: cada lote se confirma solo cuando termina, así que al volver se reintentan únicamente los lotes pendientes. Como la escritura es idempotente, nadie ve el aviso repetido. *Paso la palabra a César.*